Homework 1 (Classical ML): Predicting hERG Blockade from SMILES Codes

You will be assigned a group of 4 students based on a variety of factors. Only 1 student from the group of 4 needs to submit on Canvas.

Goal

Build and deploy a machine learning classifier that predicts whether a molecule is a hERG blocker from its SMILES string. hERG blockade is an important drug safety issue because it can contribute to cardiotoxicity risk during drug development. You will ultimately decide how to extract features for SMILES strings, what model to use, and how to deploy your model so that it is usable by others.

Dataset

Use the dataset provided in `data/hw1data.tab`, and the starter template loads it with `load_split()`:

```
split = load_split()
```

Each split contains three columns:

- `Drug_ID`: molecule identifier
- `Drug`: SMILES string
- `Y`: label, where 1 = hERG blocker and 0 = non-blocker

Environment Setup

Everyone should work from the same pinned environment so that results are comparable. Run the following commands in your terminal:

```
conda env create -f environment.yml
```

```
conda activate cdd203-hw1
```

As a backup, if you already have Python 3.10, 3.11 or 3.12, create a fresh virtual environment (`python -m venv .venv`, then activate it) and run `pip install -r requirements.txt`. **Other Python versions will not work for this assignment's list of dependencies.**

You will modify `Part1_TEMPLATE.py`: rename it to `Part1_[GROUP_NAMES].py` and fill in the four TODOs: your group member's names, your threshold, `featurize()`, and `build_model()`.

Part 1: Train and Evaluate a Classifier

Train a supervised classifier using the `Drug` column as input and `Y` as the outcome.

The provided code in `Part1_Template.py` already:

1. Loads the train/validation/test split.
2. Trains at least one classifier.
3. Evaluates performance on the test set.

First (TODO number 2), choose one decision threshold besides `0.5` and briefly explain how changing the threshold affects the tradeoff between catching true hERG blockers and falsely flagging non-blockers. In your `README.md` file (see below), describe why you chose this threshold over others given the use case of this AI model.

Next (TODO number 3), decide on a way to convert the SMILES strings into numbers usable by machine learning (i.e., to engineer the features of the data). Don't forget that the length of the numerical representation must be the same regardless of the length of the input SMILES string, and it is up to you how to handle this. In your `README.md` file (see below), describe your feature engineering strategy. Importantly, SMILES strings in the held-out test set can be up to 200 characters long, longer than any in hw1data.tab, so make sure featurize handles them (you can and should test for this explicitly before submitting).

Finally (TODO number 4), return the model that will be used to train on the data. Unless you are feeling particularly sophisticated, this should be a `sklearn` model.

The code automatically saves your model to an ONNX file if it is a `sklearn` model. However, if you don't use sklearn, write your own `save_onnx` that meets the TODO 4 requirements.

Part 2: Deploy a Simple Web App

Deploy your model as a simple public web app. You may use any reasonable deployment approach, such as Hugging Face Spaces, Streamlit Community Cloud, Render, or if particularly ambitious, a custom solution.

A recommended low-friction option is a one-file Gradio app on Hugging Face Spaces, which will allow you to easily upload and share your AI-powered web app. We learned about Gradio in class, and there are tutorials online for how to deploy a Gradio app to Hugging Face Spaces.

At a high level, your app should:

1. Load your saved model on the backend.
2. Let the user type a SMILES string.
3. Validate the SMILES string in real time. Disable the "Predict" or "Submit" button until the SMILES string is valid.

4. Convert the valid SMILES string into the correct feature space.
5. Return the predicted class and predicted probability.

The app should feel usable and responsive: validation feedback should appear quickly, error messages should be understandable, and the output should be easy for a non-expert to interpret.

Submission

Submit on Canvas a link to a GitHub repository containing:

1. Python script (.py) used to train the model, based on `Part1_TEMPLATE.py`. Name the file `Part1_[FIRST_AND_LAST_NAME_OF_GROUP_MEMBERS].py`.
2. Saved model file. We will use this to evaluate on a held-out test set that is different in composition from the test set that you have access to. Use the ONNX (Open Neural Network Exchange) file format to save the model, since this format supports models trained from various frameworks (sklearn, TensorFlow, PyTorch). Name the file `Part1_[FIRST_AND_LAST_NAME_OF_GROUP_MEMBERS].onnx`. List any added libraries in `requirements.txt`. If the model file is too large to commit, consult with Peter to work out another solution. It is unlikely that the best solution will involve having a large ONNX file, however.
3. Public web app URL in the `README.md` file. If using Hugging Face Spaces, the URL would look like `https://huggingface.co/spaces/[YOUR_USERNAME]/[YOUR_PROJECT_NAME]`. Note, however, that you are not required to deploy your web app using HuggingFace Spaces – you can use any service and workflow you'd like. Put the link to the web app in a `README.md` file in your GitHub repository.
4. Code for the deployed app, including the Hugging Face app code if using Spaces. Since the code may potentially consist of multiple files, put all code into a zipped folder named `Part 2.zip`. Copy your `.onnx` file into `Part 2.zip` as well, so the app loads the same model you submitted.
5. A short writeup (up to 3 paragraphs) explaining (a) your choice of decision threshold (TODO 2), (b) feature engineering approach (TODO 3), and (c) model selection and corresponding hyperparameters (TODO 4). Put this paragraph in the `README.md` file in your GitHub repository.
6. A short paragraph explaining how you used an LLM/generative AI to help with your assignment. Put this paragraph in the `README.md` file in your GitHub repository.

All group member names should be in the `README.md` file.

Grading

- Main grade: 94%
    - Model (submitted ONNX file) makes a prediction of 0 or 1, or a probability ranging from 0 to 1, given an input SMILES string (you can test this with the accompanying `TestModel.py` file): 10%

    - When running `python TestModel.py <your script>`, every line should say PASS: 10%
    - Running the model training code leads to the creation of a model similar in nature to the submitted ONNX model file (with slight deviations due to not setting a random seed or the stochasticity of the training process being acceptable): 10%
    - Deployed web app: 10%
    - Real-time validation and disabled submit button until valid SMILES on the deployed web app: 10%
    - The web app actually outputs the same predictions as your submitted ONNX model file (this will be verified): 25%
    - The web app has a sleek design and is not overly cluttered with non-essential visual elements: 5%
    - Writeup about modeling decisions: 9%
    - Reflection on LLM use: 5%
- Competition: 6%
    - To get the last 6%, your submitted model (i.e., your `Part1_[YOUR_FIRST-AND_LAST_NAMES].onnx` file) will be submitted to a series of competitions against every other group in the class. The various competitions are: highest sensitivity with specificity > 0.7, highest specificity with sensitivity > 0.7, highest precision with sensitivity > 0.7, highest AUROC, fastest model (including calling `featurize`) with AUROC > 0.725, and fastest model including calling `featurize`) with AUROC > 0.85 (molecules scored per second on our test set during model inference). Importantly, these competitions will be run on a held-out test set that is not available to you during the assignment, mimicking the real world use of AI models.
    - All submissions will be ranked 1st 2nd and 3rd place in all of these categories. If your highest ranking across all competitions is 1 (i.e., you got 1st place in one or more competitions), you get 6% added to your grade. If your highest ranking across all competitions is 2, you get 4% added to your grade. If your highest ranking across all competitions is 3, you get 2% added to your grade.
    - I recommend using a random seed wherever possible to ensure reproducibility. Otherwise, how your model performs in the competition will be "luck of the draw".
    - Note that UCSF final grades are A / B / C, i.e., there is no "A+" or "A-". So, the competition should not be stressful.

Restrictions

- Use the versions in `requirements.txt` and don't change them. You may add only these libraries, at exactly these versions (list them in `requirements.txt`): `rdkit==2023.9.6, torch==2.2.2, transformers==4.46.3.` To use anything

else, ask Peter first. No TensorFlow for this assignment since it is not as popular in 2026 (though Peter used it a lot in his PhD days).

- For this assignment, your code must run on a CPU: no `.cuda()` or hard-coded `"mps"` devices for this assignment only.
- If you elect to go beyond `sklearn`, you must make sure that your submission conforms to the requirements.
- No models above 150 million parameters are allowed for either feature extraction or classification.
- The training script must finish in under 30 minutes on a laptop CPU. This is a small, low-dimensional dataset: the training should be correspondingly quick. We will learn about tricks for working with high-dimensional data later.
- Anything you train yourself must go inside your ONNX file. That file must be at most 50 MB and committed directly to GitHub. No bypassing this requirement through tricks such as using Git LFS allowed.
- If using a pretrained deep learning model, it must be public on Hugging Face, loaded with `from_pretrained`, without `trust_remote_code`. If ambitious and well-resourced, you could in theory upload your own pretrained model. Name the model in your README.
- Before submitting, build a fresh environment and run both `python Part1_[GROUP_NAMES].py` and `python TestModel.py Part1_[GROUP_NAMES].py` in it. `TestModel.py` will ensure that your submission satisfies the requirements of the assignment.